



Algorithmen und Datenstrukturen

Leon Johann Brettin



Algorithmen und Datenstrukturen

6. Datenstrukturen



Datenstruktur

- Eine Datenstruktur ist eine bestimmte Art, **Daten zu verwalten** und miteinander zu verknüpfen, um in geeigneter Weise auf diese **zugreifen** und diese **manipulieren** zu können.
- Datenstrukturen sind immer mit bestimmten **Operationen** verknüpft, um eben diesen Zugriff und diese Manipulation zu ermöglichen.



Abstrakte Datenstruktur

- Geheimnisprinzip

- Der Programmierer nutzt nur zugesicherte Eigenschaften, kennt die Implementierung nicht.
- „Programming by Contract“
- Interface, Application Programming Interface – API

- Formale Spezifikation der Datenstruktur

- Eigenschaften der Datenstruktur definiert über logische Formeln, die Ergebnisse der Operationen beschreiben.
- Beispiel: Die Länge der leeren Liste ist 0.



Formalisierung Liste

```
type List(T)
import Nat
operators
  [] : → List
  _ ◦ _ : T x List → List
  head : List → T
  tail : List → List
  length : List → Nat
axioms ∀l : List, ∀x : T
  head(x ◦ l) = x
  tail(x ◦ l) = l
  length( [] ) = 0
  length( x ◦ l ) = length(l) + 1
```

- Mögliche Listen
 - []
 - 3 ◦ []
 - 5 ◦ (3 ◦ [])
 - 3 ◦ (5 ◦ (3 ◦ []))
- Abstrakte Datentypen
 - Natürliche Zahlen
 - Mengen
- Exakte Formalisierung oft schwierig



Datenstrukturen – Beispiele

- Grundlegende Datenstrukturen:
 - Kollektion, Stack, Warteschlange, Liste, ...
 - Zahlreiche mögliche Anwendungen
 - Oft Grundlage für speziellere Datenstrukturen
- Indexstrukturen:
 - Bäume und Hash-Tabellen
 - Effizienter Zugriff auf große Datenmengen
- Komplexe Netzwerke:
 - Grundlage: Graphen
 - Verschiedene interne Darstellungsformen
- Anwendungsspezifische Datenstrukturen
 - Elektrische Schaltungen, Genom, Multimedia, ...



Zielsetzungen für Datenstrukturen

- Implementierung komplexer Datentypen
 - Abbildung mit Hilfe einer Programmiersprache
 - Wiederverwendbarkeit
 - Speichereffizienz
- Implementierung der Operationen
 - Algorithmen aus Spezifikationen
 - Laufzeiteffizienz



Wiederverwendbarkeit

- Meist: (Wieder-)verwendung von Datenstrukturen
 - Nutzung existierender Implementierung in Form von Klassen, Klassenbibliotheken, Packages
 - gegebenenfalls Erweiterung durch Vererbung oder Einbettung
- Aber auch: eigenständige Implementierung
 - insbesondere für anwendungsspezifische Datenstrukturen
 - Ausnutzung der Anwendungssemantik führt meist zu effizienteren Lösungen



Motivation: Kollektionen

- Klassen zur Verwaltung einer Mehrzahl gleich strukturierter Objekte
- Ausprägungen: Felder, Listen, Stacks, Sets, . . .
- Beispiel: Realisierung als Feldtypen
 - `int[]` `feld`
- Probleme:
 - Nutzung mit verschiedenen Elementtypen: Feld von `int`-Werten, von Strings, von rationalen Zahlen, ...
 - umständliche Handhabung: Anfügen, Einfügen an Position 0, ...



Realisierungsvarianten

- Gemeinsame Wurzelklasse `Object`:
 - Elementtyp = Wurzelklasse
 - Problem: Typunsicherheit
 - Beispiel: Java
- Schablonen oder Templates
 - Instanziierung mit konkretem Elementtyp
 - typsicher
 - Generierung spezifischen Codes durch Compiler
 - Beispiel: C++, Java 5.0 (Generics)...



Beispiel: Feld von Objekten

- Implementierung auf Basis des Java-Array-Typs

```
public class ObjFeld {  
    private Object daten[] = null;  
  
    public ObjFeld() {}  
  
    public int laenge() {  
        return (daten == null ? 0 : daten.length);  
    }  
  
    public Object element(int idx) {  
        if (daten == null || idx < 0 || idx >= laenge())  
            throw new ArrayIndexOutOfBoundsException();  
        else  
            return daten[idx];  
    }  
}
```





Einfügen von Elementen

```
public void einfuegen(int pos, Object o) {  
    int i, num = laenge();  
    if (pos == -1)  
        pos = num;  
    Object ndaten[] = new Object[num + 1];  
    for (i = 0; i < pos; i++)  
        ndaten[i] = daten[i];  
    ndaten[i++] = o;  
    for (; i < num + 1; i++)  
        ndaten[i] = daten[i - 1];  
    daten = ndaten;  
}
```





Einfügen von Elementen

```
public void einfuegen2(int pos, Object o) {  
    int num = laenge();  
    if (pos == -1)  
        pos = num;  
    Object ndaten[] = new Object[num + 1];  
    System.arraycopy(daten, 0, ndaten, 0, pos);  
    ndaten[pos] = o;  
    System.arraycopy(daten, pos, ndaten, pos + 1, num - pos);  
    daten = ndaten;  
}
```





Einfügen eines Objekts

- Objekte der Klasse RNumber (Rational Number) einfügen

```
ObjFeld of = new ObjFeld();  
of.einfuegen(-1, new RNumber(1,3));  
of.einfuegen(-1, new RNumber(2,3));
```



- Gemischte Objekte sind (hier) zulässig

```
of.einfuegen(2, Integer.parseInt("42"));
```





Typkonvertierung und Test

- Zugriff auf Elemente erfordert Typkonvertierung (type cast)

```
RNumber rn = (RNumber)of.element(1);
```



- Test auf Zugehörigkeit zu einer Klasse

```
Object o = of.element(2);  
if (o instanceof RNumber) {  
    RNumber r = (RNumber) o;  
} else if (o instanceof Integer) {  
    Integer i = (Integer) o;  
}
```





Java Generics

```
public class GenericFeld<T> {  
    private T daten[] = null;  
  
    public GenericFeld() { }  
  
    public int laenge() {  
        return (daten == null ? 0 : daten.length);  
    }  
  
    public T element(int idx) {  
        if (daten == null || idx < 0 || idx >= laenge())  
            throw new ArrayIndexOutOfBoundsException();  
        else  
            return daten[idx];  
    }  
}
```



```
public void einfuegen2(int pos, T o) {  
    int num = laenge();  
    if (pos == -1)  
        pos = num;  
    T ndaten[] = (T[]) new Object[num + 1];  
    System.arraycopy(...);  
    ndaten[pos] = o;  
    System.arraycopy(...);  
    daten = ndaten;  
}  
  
public static void main(String[] args) {  
    GenericFeld<RNumber> of = new  
                                GenericFeld<RNumber>();  
    of.einfuegen2(-1, new RNumber(1, 3));  
    RNumber rn = of.element(1);  
}  
}
```





Datenstrukturen – Beispiele

- Grundlegende Datenstrukturen:
 - Kollektion, **Stack**, Warteschlange, Liste, ...
 - Zahlreiche mögliche Anwendungen
 - Oft Grundlage für speziellere Datenstrukturen
- Indexstrukturen:
 - Bäume und Hash-Tabellen
 - Effizienter Zugriff auf große Datenmengen
- Komplexe Netzwerke:
 - Grundlage: Graphen
 - Verschiedene interne Darstellungsformen
- Anwendungsspezifische Datenstrukturen
 - Elektrische Schaltungen, Genom, Multimedia, ...



Stapel (Stack) als Datentyp

- `void push(Object obj)` legt das Objekt `obj` als oberstes Element auf dem Stapel ab
- `Object pop()` nimmt das oberste Element vom Stapel und gibt es zurück
- `Object top()` gibt das oberste Element des Stapels zurück, ohne es zu entfernen
- `boolean isEmpty()` liefert `true`, wenn keine Elemente auf dem Stapel liegen, andernfalls `false`





Stapel (Stack)

```
public interface Stack {  
    public void push(Object obj) throws StackException;  
    public Object pop() throws StackException;  
    public Object top() throws StackException;  
    public boolean isEmpty();  
}  
  
public class StackException extends Exception {  
    public StackException(String msg) { super(msg); }  
    public StackException() { }  
}
```





Stapel (Stack) Generics



```
public interface Stack<T> {  
    public void push(T obj) throws StackException;  
    public T pop() throws StackException;  
    public T top() throws StackException;  
    public boolean isEmpty();  
}  
  
public class StackException extends RuntimeException {  
    public StackException(String msg) { super(msg); }  
    public StackException() { }  
}
```



Nutzung des Stapels

```
Stack stack = new Stack();
try {
    // Elemente auf dem Stapel ablegen
    stack.push("Eins");
    stack.push("Zwei");
    stack.push("Drei");
    // Elemente vom Stapel lesen
    while (!stack.isEmpty()) {
        String s = (String) stack.pop();
        System.out.println(s);
    }
} catch (StackException exc) {
    System.out.println("Fehler:" + exc);
}
```





Implementierung

- verschiedene Implementierungen möglich
- hier über statisches Feld mit vorgegebener Kapazität
- notwendig: Behandlung des Überlaufs (StackException)

```
public class ArrayStack implements Stack {  
  
    private Object[] elements = null; // Elemente  
    private int num = 0; // aktuelle Anzahl  
  
    // Stack mit definierter Kapazität  
    public ArrayStack(int size) { elements = new Object[size]; }  
  
    // Stack mit Standardkapazität  
    public ArrayStack() { this(20); }  
}
```





push() und pop()



```
public void push(Object obj) throws StackException {  
    if (num == elements.length) { // Kapazität erschöpft  
        throw new StackException();  
    }  
    elements[num++] = obj;  
}  
  
public Object pop() throws StackException {  
    if (isEmpty()) { // Stack ist leer  
        throw new StackException();  
    }  
    Object o = elements[--num];  
    elements[num] = null; // nicht behalten!  
    return o;  
}
```



top() und isEmpty()

```
public Object top() throws StackException {  
    if (isEmpty()) { // Stack ist leer  
        throw new StackException();  
    }  
    return elements[num - 1];  
}  
  
public boolean isEmpty() {  
    return num == 0;  
}
```





Verbesserungen von ArrayStack

- Unbegrenzte Stapelgröße?
 - Listen
 - Neuen Speicher reservieren und umkopieren
 - Wie viel mehr reservieren?
- hierarchische Datenstruktur
 - Beliebig erweiterbar ohne Umkopieren?



Stack mit verketteter Liste

- Anstelle des bei `ArrayStack` verwendeten Feldes wird ein `List`-Objekt zur Speicherung der Stack-Elemente verwendet.
- Welches Ende der Liste soll die Spitze des Stack implementieren?
 - Da es nur konstanten Aufwand erfordert, am Anfang der Liste Objekte hinzuzufügen und zu entfernen, wählen wir den Anfang der Liste als Spitze des Stack.



Klasse ListStack

```
public class ListStack implements Stack {  
  
    private List list; // Liste zur Verwaltung der Elemente  
  
    public ListStack() {  
        list = new List();  
    }  
  
    @Override  
    public void push(Object obj) throws StackException {  
        list.addFirst(obj);  
    }  
  
    @Override  
    public Object pop() throws StackException {  
        if (isEmpty()) {  
            throw new StackException();  
        }  
        return list.removeFirst();  
    }  
}
```

```
@Override  
public Object top() throws StackException {  
    if (isEmpty()) {  
        throw new StackException();  
    }  
    return list.getFirst();  
}  
  
@Override  
public boolean isEmpty() {  
    return list.isEmpty();  
}  
}
```



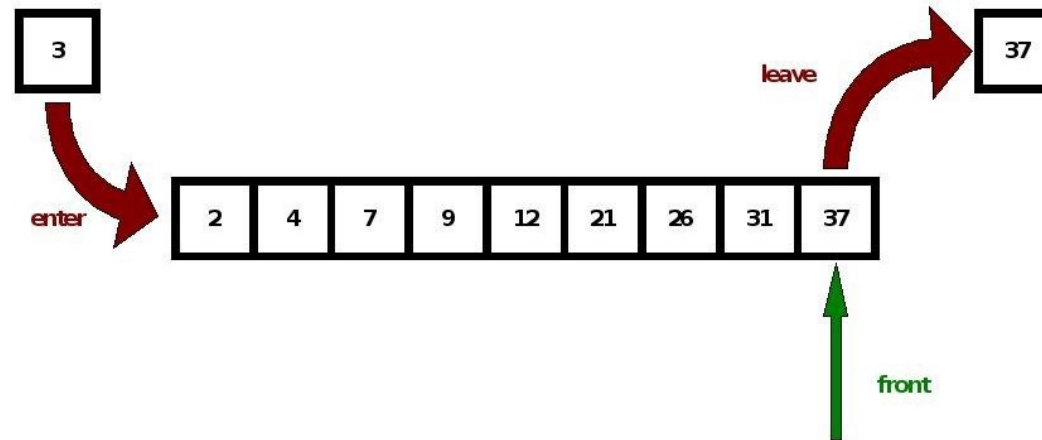
Datenstrukturen – Beispiele

- Grundlegende Datenstrukturen:
 - Kollektion, Stack, Warteschlange, Liste, ...
 - Zahlreiche mögliche Anwendungen
 - Oft Grundlage für speziellere Datenstrukturen
- Indexstrukturen:
 - Bäume und Hash-Tabellen
 - Effizienter Zugriff auf große Datenmengen
- Komplexe Netzwerke:
 - Grundlage: Graphen
 - Verschiedene interne Darstellungsformen
- Anwendungsspezifische Datenstrukturen
 - Elektrische Schaltungen, Genom, Multimedia, ...



FIFO – Warteschlange – Queue

- FIFO: First-in-first-out
 - Stapel ist LIFO: Last-in-first-out
 - Warteschlange: engl. Queue





FIFO – Warteschlange – Queue

- `void enter(Object obj)` fügt das Objekt `obj` in die Queue ein
- `Object leave()` nimmt das älteste Objekt aus der Queue und gibt es zurück
- `boolean isEmpty()` liefert `true`, wenn keine Elemente in der Queue vorhanden sind, andernfalls `false`
- `Object front()` gibt das älteste Element der Queue zurück, ohne es zu entfernen



Implementierung der Queue

- Implementierung:
 - Ein Zeiger u (für upper) markiert die erste freie Position im Feld
 - Das erste Element befindet sich immer auf Position 0. Dann müssen nach Herausnehmen eines Elements die anderen Elemente verschoben werden.
- Alternative:
 - Ein zweiter Zeiger l (für lower) markiert die Position des ersten Elements
 - Die beiden Zeiger wandern zirkulär über das Feld
 - Ringpuffer
 - begrenzte Kapazität



Implementierung Queue 1



```
public class ArrayQueue implements Queue {  
    private Object[] elements; // Elemente  
    private int l = 0, u = 0; // unterer und oberer Zeiger  
  
    // Queue mit vorgegebener Kapazität erzeugen  
    public ArrayQueue(int size) {  
        elements = new Object[size];  
    }  
  
    // Queue mit Standardkapazität erzeugen  
    public ArrayQueue() { this(100); }
```



Implementierung Queue 2

```
public void enter(Object obj) throws QueueException {  
    if (u == (l - 1 + elements.length) % elements.length) {  
        // Kapazität ist erschöpft  
        throw new QueueException();  
    }  
    elements[u] = obj;  
    u = (u + 1) % elements.length; // oberen Zeiger aktualisieren  
}  
  
public Object leave() throws QueueException {  
    if (isEmpty())  
        throw new QueueException();  
    Object obj = elements[l];  
    elements[l] = null;  
    l = (l + 1) % elements.length; // unteren Zeiger aktualisieren  
    return obj;  
}
```



Implementierung Queue 3

```
public Object front() throws QueueException {  
    if (isEmpty())  
        throw new QueueException();  
    return elements[l];  
}  
  
public boolean isEmpty() {  
    return l == u;  
}
```





Queue Generic

```
public interface GQueue<T> {  
    public void enter(T obj) throws QueueException;  
    public T leave() throws QueueException;  
    public T front() throws QueueException;  
    public boolean isEmpty();  
}  
  
public class ArrayGQueue<T> implements GQueue<T> {  
    private T[] elements; // Elemente  
    private int l = 0, u = 0; // unterer und oberer Zeiger  
  
    @SuppressWarnings("unchecked")  
    public ArrayGQueue(int size) { elements = (T[]) new Object[size]; }  
  
    public void enter(T obj) throws QueueException {  
        // ...  
    } // ...  
}
```



Komplexität kommt
im nächsten
Foliensatz!

Bewertung der Implementierung

- Zeit-Komplexität aller Methoden $O(1)$, d.h. sie haben konstante Laufzeit
- Speicherplatzbedarf wird durch Initialisierung mit Größe N vorgegeben.
Speicherplatz-Komplexität $O(N)$
- Problem der Implementierungen ist die Größenbeschränkung des Feldes.
- Ausweg wäre das Umkopieren der Elemente in ein größeres Feld
 - Dann Aufwand beim Einfügen manchmal $O(N)$
 - amortisiert (verteilt auf alle Aufrufe): $O(1)$
- Vorteile von Feldern ist der geringe Speicherplatzbedarf.



Dynamische Erweiterung

```
public void enter(Object obj) throws QueueException {  
    if (u == (1 - 1 + elements.length) % elements.length) {  
        // Kapazität ist erschöpft  
        extend();  
        // ...  
    }  
}
```





Dynamische Erweiterung

```
private void extend() { // Wenn feste Kapazität nicht gewünscht
    Object[] newElements = new Object[elements.length * 2];
    if (u < l) {
        // kopiere erst von l bis rechtes Ende, dann unteren Teil
        System.arraycopy(elements, l, newElements, 0, elements.length - l);
        System.arraycopy(elements, 0, newElements, elements.length - l, u);
        u = elements.length - l + u;
    } else {
        System.arraycopy(elements, l, newElements, 0, u - l);
        u = u - l;
    }
    elements = newElements;
    l = 0;
}
```





Zusammenfassung

- Grundlegende Datenstrukturen:
 - Kollektion
 - Ansammlung von Elementen ohne besondere Reihenfolge
 - Stapel
 - Last-in-first-out: Nur zuletzt eingefügtes Element ist verfügbar.
 - Warteschlange
 - First-in-first-out: Wer zuerst angekommen war, wird als nächstes bedient/bearbeitet/entnommen.
- Implementierung mit Feldern
 - Eventuell Umkopieren des Feldes nötig.



Welche Ausgabe erwarten Sie?

```
public class StackTest2 {  
    public static void main(String args[]) {  
        GStack<String> stack = new ArrayStackG<String>(4);  
        try {  
            // Elemente auf dem Stapel ablegen  
            stack.push("A");  
            stack.push("H");  
            stack.push("A");  
            stack.push("L");  
            readAndEmpty(stack);  
            // Elemente auf dem Stapel ablegen  
            stack.push("H");  
            stack.push("A");  
            stack.push("L");  
            stack.push("T");  
            stack.push("!");  
            readAndEmpty(stack);  
        } catch (StackException exc) {  
            System.out.println("Fehler:" + exc);  
        }  
    }  
    public static <T> void readAndEmpty(GStack<T> stack) {  
        try { // Elemente vom Stapel lesen  
            while (!stack.isEmpty()) {  
                String s = stack.pop().toString();  
                System.out.print(s);  
            }  
        } catch (StackException exc) {  
            System.out.println("Fehler:" + exc);  
        }  
        System.out.print("--");  
    }  
}
```

- A. AHAL--HALT!--
- B. !TLAH--LAHA--
- C. LAHA-- Fehler--
- D. AHAL-- Fehler--
- E. LAHA--!TLAH--



Welche Datenstruktur ist das?

- Es gibt die Operationen
 - push, pop, top, length, isEmpty

A. Kollektion

B. Stack

C. Queue

D. Array

E. Tree



Welche Datenstruktur ist das?

- Es gibt die Operationen
 - length, isEmpty, enter, leave, front

A. Kollektion

B. Stack

C. Queue

D. Array

E. Tree



Welche Datenstruktur ist das?

- Es gibt die Operationen
 - length, set(i,x), get(i)

A. Kollektion

B. Stack

C. Queue

D. Array

E. Tree



Queue

- Bei einer Warteschlange (Queue) gilt:
 - 1) Einfügen und Entnehmen geschehen an der gleichen Seite
 - 2) Einfügen und Entnehmen geschehen an verschiedenen Seiten
 - 3) Jeder Speicherplatz kann (über die Zeit) verschiedene Elemente enthalten.

- A. 1)
- B. 1) und 2)
- C. 2)
- D. 2) und 3)
- E. 3)
- F. Keins davon